

MECHATRONICS AND IOT ASSIGNMENT REPORT-5

**Design and simulate a Smart Flood Level Monitoring System
using Ultrasonic Sensor**

TEAM MEMBERS:

KOWSHIK K 24MER025

NITHISH J 24MER037

KAVIN KRISHNA D S - 24MER024

LOGESH S 24MER027

1. Project Overview

Flooding is one of the most common natural disasters and can cause significant loss of life, property, agriculture, and infrastructure. Early detection of rising water levels can provide valuable time for authorities and people to take preventive action.

The proposed **Smart Flood Level Monitoring System** continuously measures the distance between an ultrasonic sensor and the water surface. A microcontroller processes the measured distance and calculates the corresponding water level. Depending on the water level, the system indicates **Normal, Warning, or Critical Flood** conditions using LEDs, an LCD display, and a buzzer.

The prototype can be simulated using platforms such as **Wokwi, Proteus, or Tinkercad**, and can subsequently be implemented using an Arduino Uno, HC-SR04 ultrasonic sensor, LEDs, buzzer, and LCD.

Main objectives

- Continuously monitor the water level.
- Detect abnormal and critical increases in water level.
- Provide immediate visual and audible alerts.
- Display the measured water level and status.
- Develop a low-cost prototype suitable for disaster-management applications.
- Provide a system that can be extended with IoT/GSM communication for remote alerts.

2. Problem Statement

Traditional flood monitoring systems may depend on manual measurement or observation of rivers, drains, reservoirs, and low-lying areas. Such methods can be slow, labor-intensive, and unsuitable for continuous monitoring.

A sudden rise in water level may occur due to heavy rainfall, dam discharge, drainage blockage, or river overflow. If the increase is not detected quickly, there may not be sufficient time to issue warnings and evacuate people.

Therefore, there is a need for an **automatic, continuous, low-cost flood-level monitoring system** capable of detecting dangerous water levels and generating alerts in real time.

3. Proposed Solution

The proposed system is an **automatic water-level monitoring and early-warning system** designed to continuously monitor the water level in a reservoir, lake, dam, or other water body. The system uses an **HC-SR04 ultrasonic sensor** mounted above the water surface to measure the distance between the sensor and the water. Since the sensor does not need to touch the water, it provides a simple and contactless method of monitoring.

The **HC-SR04** transmits ultrasonic waves toward the water surface. These waves are reflected back to the sensor, and the time taken for the echo to return is measured by the **Arduino UNO**. Using the measured time, the Arduino calculates the distance between the sensor and the water surface. As the water level rises, this distance decreases.

The Arduino compares the measured distance with predefined water-level limits and classifies the condition into different safety levels:

- **Safe Level:** Water level is within the normal operating range. The **Green LED** remains ON.
- **Warning Level:** Water level is approaching the critical limit. The **Yellow LED** turns ON to indicate that attention is required.
- **Critical Level:** Water level reaches a dangerous limit. The **Red LED and Buzzer** are activated to provide an immediate warning.

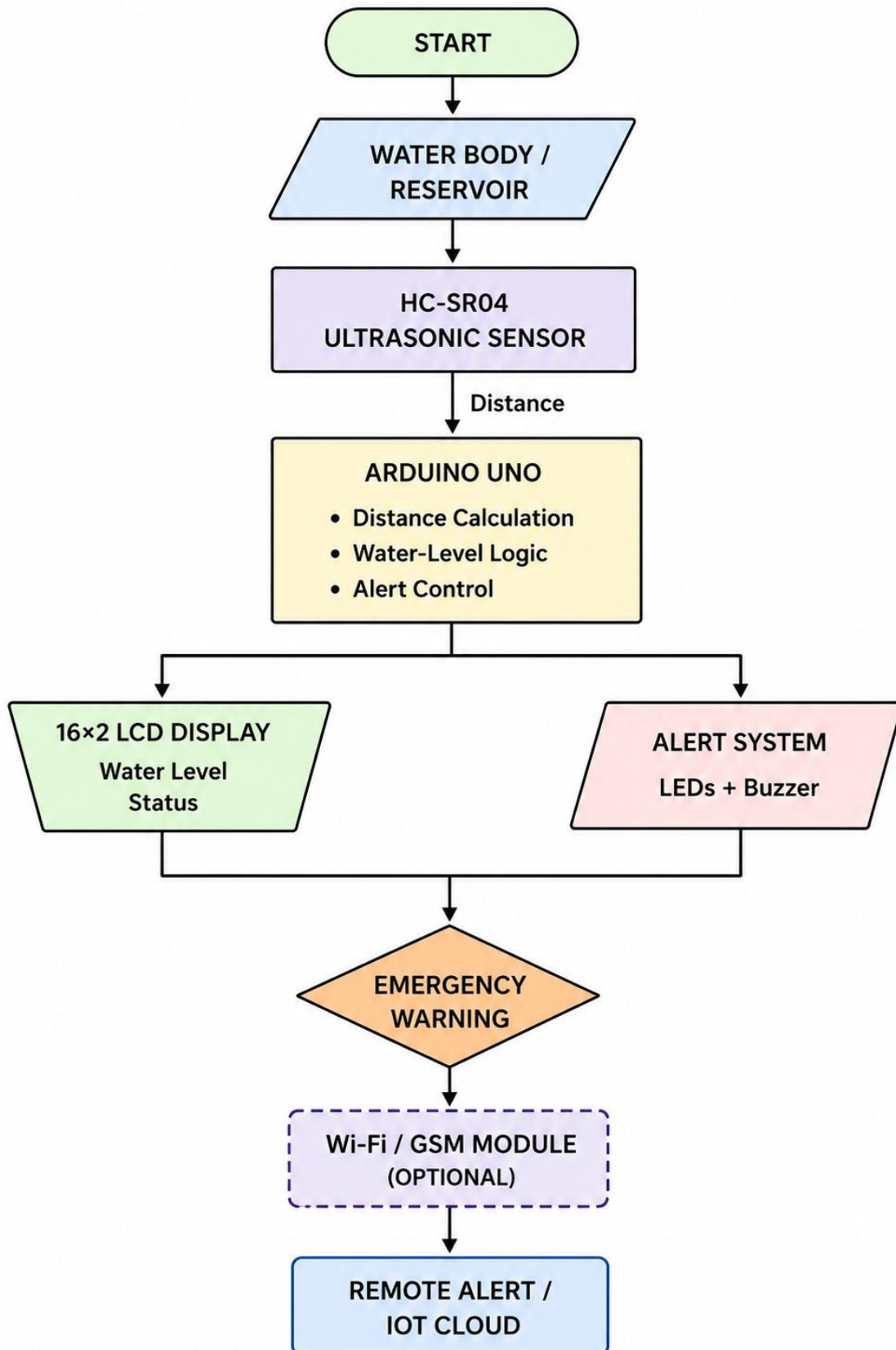
The current water-level status is displayed on a **16×2 LCD**, allowing nearby users or operators to easily monitor the condition.

The remote system can provide information such as:

- Current water level
- Water-level status
- Warning or critical alert
- Date and time of the event
- Sensor status
- Historical water-level data

Overall, the proposed system provides a **low-cost, automatic, contactless, and real-time water-level monitoring solution**. It reduces the need for continuous manual observation and provides early warnings that can help improve safety and disaster preparedness.

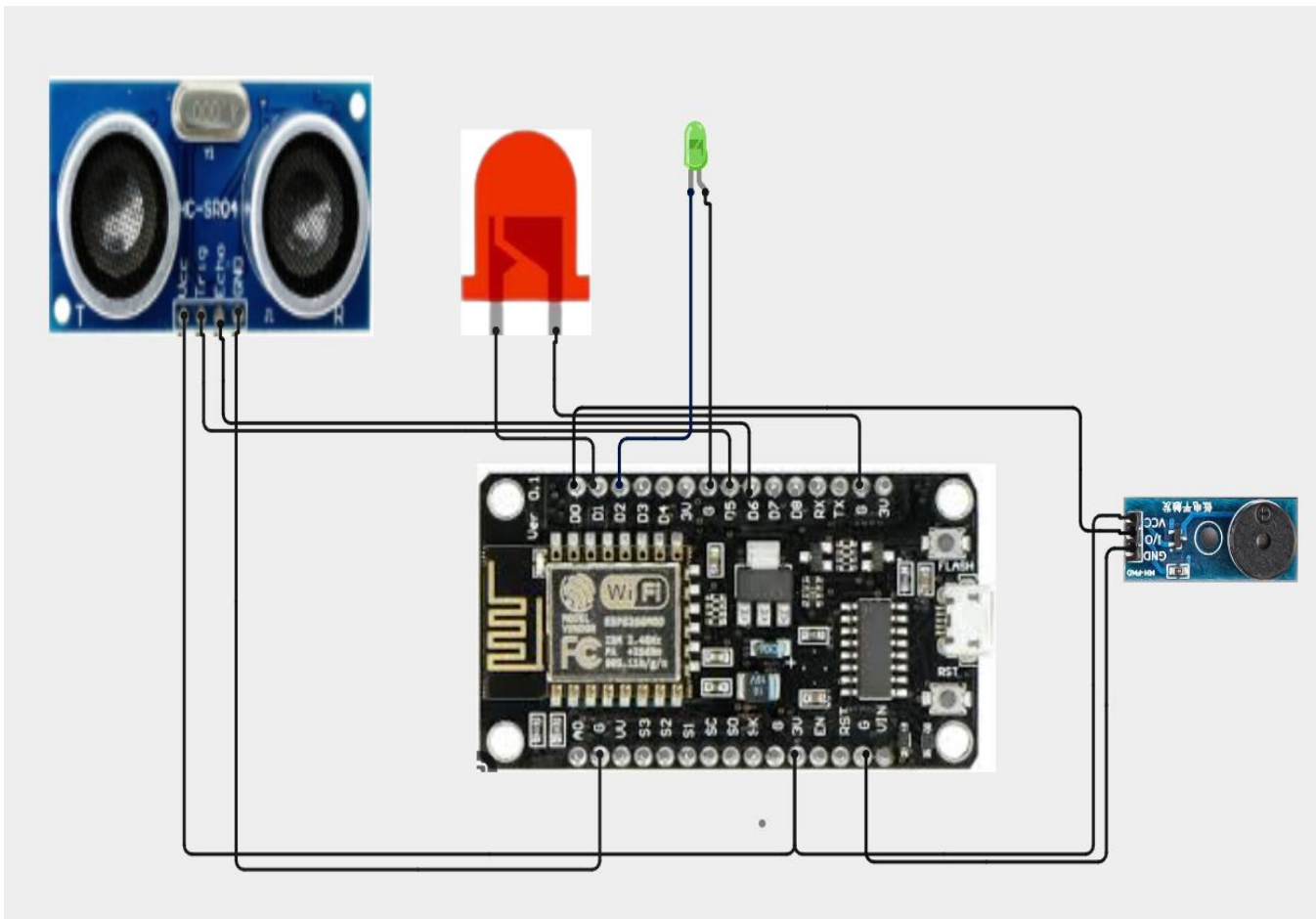
4. System Architecture



5. Circuit Table

Component	Pin	Arduino Connection	Function
HC-SR04	VCC	5V	Sensor power
HC-SR04	GND	GND	Ground
HC-SR04	TRIG	D9	Sends ultrasonic pulse
HC-SR04	ECHO	D10	Receives reflected pulse
Green LED	Anode	D2 through 220 Ω	Normal indication
Yellow LED	Anode	D3 through 220 Ω	Warning indication
Red LED	Anode	D4 through 220 Ω	Critical indication
Buzzer	Positive	D5	Audible alarm
Buzzer	Negative	GND	Ground
LCD 16×2	VCC	5V	LCD power
LCD 16×2	GND	GND	Ground
LCD RS	RS	D12	Register selection
LCD EN	EN	D11	Enable control
LCD D4	D4	D6	LCD data
LCD D5	D5	D7	LCD data
LCD D6	D6	D8	LCD data
LCD D7	D7	D13	LCD data
LCD V0	Contrast	Potentiometer	Contrast adjustment

6. Circuit Design



7. Algorithm

- **Step-by-step algorithm**
- Start the system.
- Initialize the ultrasonic sensor, LCD, LEDs, and buzzer.
- Define the tank height and flood-level thresholds.
- Send a trigger pulse from the HC-SR04.
- Measure the echo pulse duration.
- Calculate the distance to the water surface.
- Calculate water level using:

$$\text{Water Level} = \text{Tank Height} - \text{Water Surface Distance}$$

- Display the water level on the LCD.
- Compare the water level with the predefined thresholds.
- If the level is normal, turn ON the green LED.

- If the level reaches the warning range, turn ON the yellow LED.
- If the level reaches the critical range, turn ON the red LED and buzzer.
- Repeat the measurement continuously.
- In an IoT-enabled version, transmit the critical-level information to a remote monitoring system.

8. Arduino Coding

```
// =====
// SMART FLOOD LEVEL MONITORING SYSTEM
// ESP8266 + HC-SR04 + 2 LEDs + Buzzer
// + WiFi + ThingSpeak + Lamp Indicator
// =====

#include <ESP8266WiFi.h>
#include <ThingSpeak.h>

// =====
// WIFI DETAILS
// =====

#define WIFI_SSID    "Luky003"
#define WIFI_PASSWORD "kowshik03"

// =====
// THINGSPEAK DETAILS
// =====

unsigned long CHANNEL_ID = 3459014;

const char* WRITE_API_KEY = "HPFMS1KQ7C86UZFC";

WiFiClient client;

// =====
// PIN DEFINITIONS
// =====

#define TRIG_PIN  14  // D5 - GPIO14
#define ECHO_PIN  12  // D6 - GPIO12

#define GREEN_LED  5   // D1 - GPIO5
#define RED_LED    4   // D2 - GPIO4

#define BUZZER     0   // D3 - GPIO0
```

```

// =====
// BUZZER SETTINGS
// Active LOW buzzer
// =====

#define BUZZER_ON LOW
#define BUZZER_OFF HIGH

// =====
// TANK SETTINGS
// =====

const float TANK_HEIGHT = 30.0; // cm

// =====
// THINGSPEAK UPDATE INTERVAL
// =====

unsigned long lastSendTime = 0;

const unsigned long SEND_INTERVAL = 15000;

// =====
// SETUP
// =====

void setup() {

  Serial.begin(115200);

  delay(1000);

  // -----
  // SENSOR
  // -----

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  // -----
  // LEDs
  // -----

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

```

```

// -----
// BUZZER
// -----

pinMode(BUZZER, OUTPUT);

// -----
// INITIAL CONDITION
// -----

digitalWrite(TRIG_PIN, LOW);

digitalWrite(GREEN_LED, LOW);
digitalWrite(RED_LED, LOW);

digitalWrite(BUZZER, BUZZER_OFF);

// -----
// SERIAL START
// -----

Serial.println();
Serial.println("=====");
Serial.println("  SMART FLOOD LEVEL MONITORING");
Serial.println("  ESP8266 + HC-SR04 + ThingSpeak");
Serial.println("=====");

// -----
// WIFI CONNECTION
// -----

Serial.print("Connecting to WiFi");

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

while (WiFi.status() != WL_CONNECTED) {

  Serial.print(".");
  delay(500);
}

Serial.println();
Serial.println("WiFi Connected!");

Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

```

```

// -----
// THINGSPEAK
// -----

ThingSpeak.begin(client);

Serial.println("ThingSpeak Started");
Serial.println("-----");
}

// =====
// LOOP
// =====

void loop() {

    // =====
    // CHECK WIFI
    // =====

    if (WiFi.status() != WL_CONNECTED) {

        Serial.println("WiFi disconnected!");

        WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

        delay(1000);

        return;
    }

    // =====
    // TRIGGER HC-SR04
    // =====

    digitalWrite(TRIG_PIN, LOW);

    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);

    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

    // =====
    // READ ECHO

```

```

// =====

long duration = pulseIn(
  ECHO_PIN,
  HIGH,
  30000
);

// =====
// SENSOR ERROR
// =====

if (duration == 0) {

  Serial.println("ERROR: No ultrasonic signal!");

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);

  digitalWrite(BUZZER, BUZZER_OFF);

  delay(1000);

  return;
}

// =====
// CALCULATE DISTANCE
// =====

float distance =
  duration * 0.0343 / 2.0;

// =====
// CHECK VALID DISTANCE
// =====

if (distance < 2.0 || distance > 400.0) {

  Serial.print("Invalid Distance: ");

  Serial.print(distance, 2);

  Serial.println(" cm");

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);

```

```

digitalWrite(BUZZER, BUZZER_OFF);

delay(1000);

return;
}

// =====
// CALCULATE WATER LEVEL
// =====

float waterLevel =
    TANK_HEIGHT - distance;

// Limit water level

if (waterLevel < 0) {

    waterLevel = 0;
}

if (waterLevel > TANK_HEIGHT) {

    waterLevel = TANK_HEIGHT;
}

// =====
// CALCULATE PERCENTAGE
// =====

float levelPercentage =
    (waterLevel / TANK_HEIGHT) * 100.0;

// Limit percentage

if (levelPercentage < 0) {

    levelPercentage = 0;
}

if (levelPercentage > 100) {

    levelPercentage = 100;
}

// =====

```

```
// VARIABLES
```

```
// =====
```

```
String floodStatus;
```

```
int floodStatusValue = 0;
```

```
int greenLight = 0;
```

```
int redLight = 0;
```

```
int buzzerStatus = 0;
```

```
int lampStatus = 0;
```

```
// =====
```

```
// LAMP INDICATOR
```

```
//
```

```
// 70% OR ABOVE = ON
```

```
// BELOW 70% = OFF
```

```
// =====
```

```
if (levelPercentage >= 70) {
```

```
    lampStatus = 1;
```

```
} else {
```

```
    lampStatus = 0;
```

```
}
```

```
// =====
```

```
// NORMAL
```

```
// 0% - 59%
```

```
// =====
```

```
if (levelPercentage < 60) {
```

```
    digitalWrite(GREEN_LED, HIGH);
```

```
    digitalWrite(RED_LED, LOW);
```

```
    digitalWrite(BUZZER, BUZZER_OFF);
```

```
    floodStatus = "NORMAL";
```

```
    floodStatusValue = 0;
```

```

greenLight = 1;

redLight = 0;

buzzerStatus = 0;

Serial.println("STATUS: NORMAL");
}

// =====
// WARNING
// 60% - 74%
// =====

else if (levelPercentage < 75) {

    digitalWrite(GREEN_LED, LOW);

    digitalWrite(RED_LED, HIGH);

    // Buzzer OFF during warning

    digitalWrite(BUZZER, BUZZER_OFF);

    floodStatus = "WARNING";

    floodStatusValue = 1;

    greenLight = 0;

    redLight = 1;

    buzzerStatus = 0;

    Serial.println("STATUS: WARNING");

    // -----
    // 70% LAMP
    // -----

    if (levelPercentage >= 70) {

        Serial.println("THINGSPEAK LAMP: ON");

    }
}

```

```

// =====
// CRITICAL
// 75% - 100%
// =====

else {

    digitalWrite(GREEN_LED, LOW);

    digitalWrite(RED_LED, HIGH);

    // BUZZER ON

    digitalWrite(BUZZER, BUZZER_ON);

    floodStatus = "CRITICAL FLOOD";

    floodStatusValue = 2;

    greenLight = 0;

    redLight = 1;

    buzzerStatus = 1;

    Serial.println("STATUS: CRITICAL FLOOD!");

    Serial.println("BUZZER: ON");

    Serial.println("THINGSSPEAK LAMP: ON");
}

// =====
// SERIAL MONITOR
// =====

Serial.println("-----");

Serial.print("Distance    : ");

Serial.print(distance, 2);

Serial.println(" cm");

Serial.print("Water Level  : ");

```

```
Serial.print(waterLevel, 2);

Serial.println(" cm");

Serial.print("Level      : ");

Serial.print(levelPercentage, 2);

Serial.println(" %");

Serial.print("Flood Status  : ");

Serial.println(floodStatus);

Serial.print("Green Light   : ");

Serial.println(
  greenLight ? "ON" : "OFF"
);

Serial.print("Red Light    : ");

Serial.println(
  redLight ? "ON" : "OFF"
);

Serial.print("Buzzer       : ");

Serial.println(
  buzzerStatus ? "ON" : "OFF"
);

Serial.print("Lamp Indicator : ");

Serial.println(
  lampStatus ? "ON" : "OFF"
);

Serial.println("-----");

// =====
// SEND DATA TO THINGSPEAK
// =====

if (millis() - lastSendTime >= SEND_INTERVAL) {

  lastSendTime = millis();
```

```
Serial.println("Sending data to ThingSpeak...");
```

```
// -----  
// FIELD 1  
// WATER LEVEL  
// -----
```

```
ThingSpeak.setField(  
    1,  
    levelPercentage  
);
```

```
// -----  
// FIELD 2  
// DISTANCE  
// -----
```

```
ThingSpeak.setField(  
    2,  
    distance  
);
```

```
// -----  
// FIELD 3  
// FLOOD STATUS  
//  
// 0 = NORMAL  
// 1 = WARNING  
// 2 = CRITICAL  
// -----
```

```
ThingSpeak.setField(  
    3,  
    floodStatusValue  
);
```

```
// -----  
// FIELD 4  
// GREEN LED  
//  
// 0 = OFF  
// 1 = ON  
// -----
```

```
ThingSpeak.setField(  
    4,
```

```
    greenLight
);

// -----
// FIELD 5
// RED LED
//
// 0 = OFF
// 1 = ON
// -----

ThingSpeak.setField(
    5,
    redLight
);

// -----
// FIELD 6
// BUZZER
//
// 0 = OFF
// 1 = ON
// -----

ThingSpeak.setField(
    6,
    buzzerStatus
);

// -----
// FIELD 7
// LAMP INDICATOR
//
// 0 = OFF
// 1 = ON
//
// ON WHEN WATER >= 70%
// -----

ThingSpeak.setField(
    7,
    lampStatus
);

// -----
// SEND ALL FIELDS
// -----
```

```

int response =
  ThingSpeak.writeFields(
    CHANNEL_ID,
    WRITE_API_KEY
  );

// -----
// CHECK RESPONSE
// -----

if (response == 200) {

  Serial.println(
    "ThingSpeak update successful!"
  );

} else {

  Serial.print(
    "ThingSpeak update failed. HTTP error: "
  );

  Serial.println(response);
}

Serial.println("-----");
}

// =====
// LOOP DELAY
// =====

delay(1000);
}

```

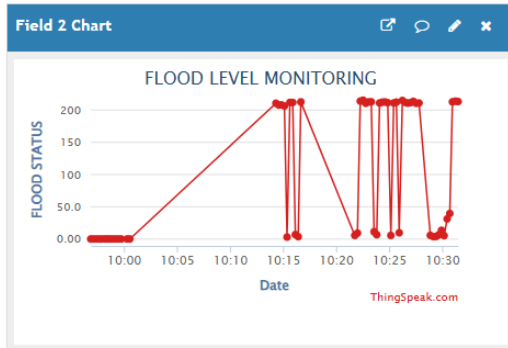
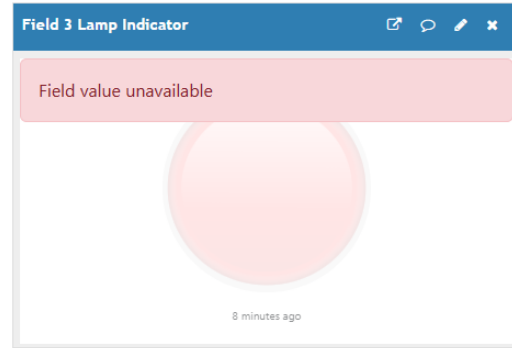
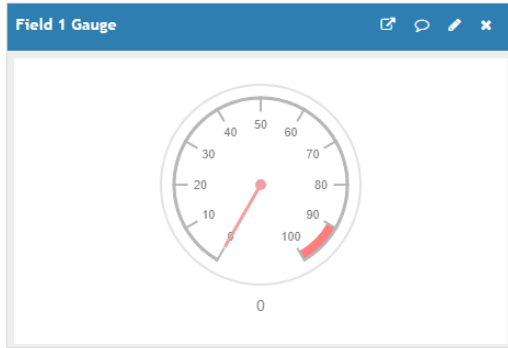
9. System Working

Channel Stats

Created: about 23 hours ago

Last entry: 8 minutes ago

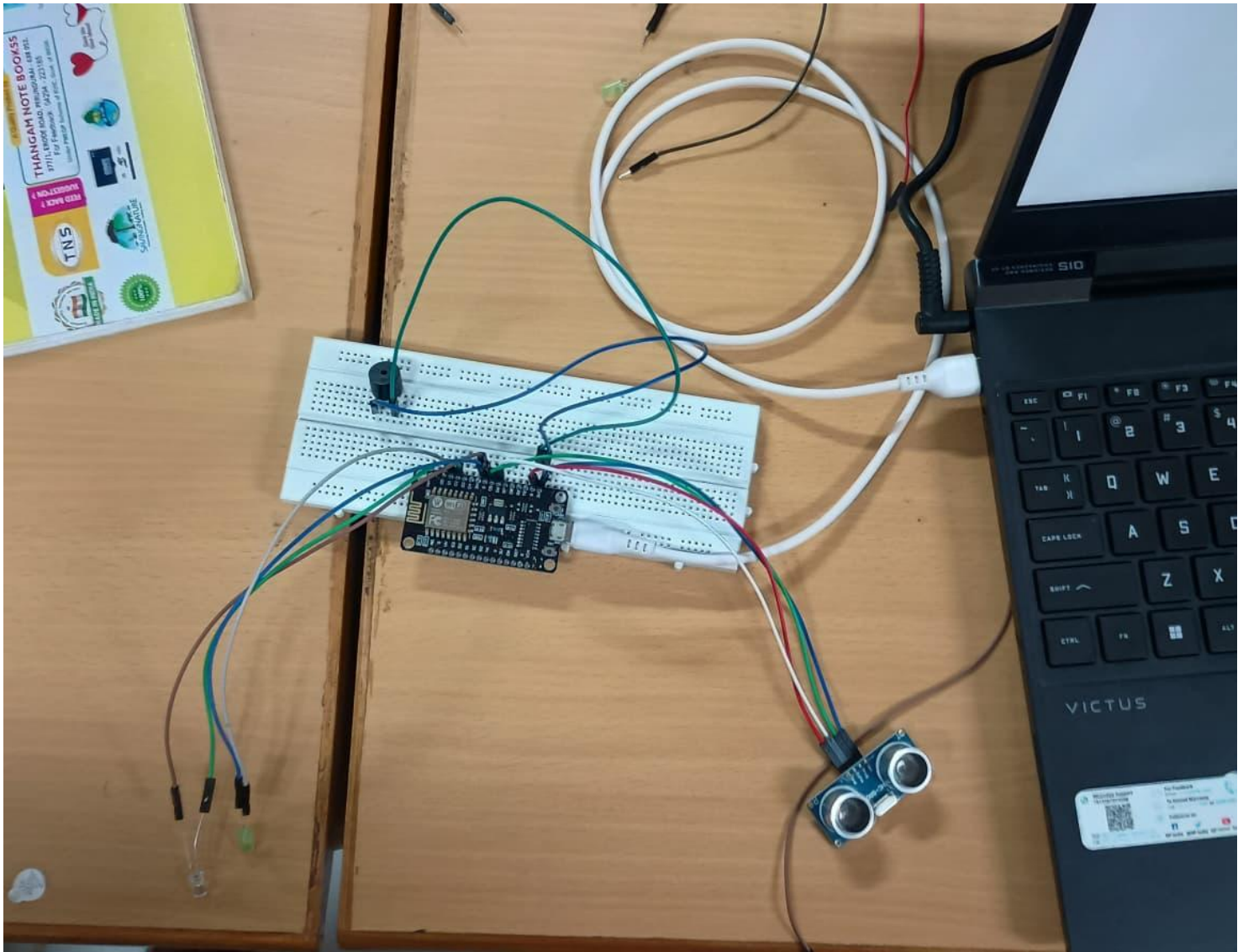
Entries: 67



10. Bill of Materials

S.No.	Component	Quantity	Approx. Purpose
1	Arduino UNO	1	Main controller
2	HC-SR04 Ultrasonic Sensor	1	Water-level measurement
3	16×2 LCD	1	Display
4	Green LED	1	Normal indication
5	Yellow LED	1	Warning indication
6	Red LED	1	Critical indication
7	Buzzer	1	Audible alarm
8	220 Ω Resistor	3	LED current limiting
9	Potentiometer, 10 k Ω	1	LCD contrast
10	Breadboard	1	Prototype assembly
11	Jumper wires	As required	Connections
12	5 V USB power supply	1	Power source
13	Water container/tank	1	Demonstration model

11. Prototype Implementation



12. Result and Performance Analysis

- **Continuous Monitoring:** The system continuously measures the water level at regular intervals.
- **Fast Response:** The ultrasonic sensor detects changes in water level quickly and the controller activates alerts immediately when thresholds are crossed.
- **Good Measurement Accuracy:** The HC-SR04 provides centimeter-level distance measurement under suitable environmental conditions.
- **Automatic Flood Detection:** The system automatically identifies **Normal, Warning, and Critical** water-level conditions without human intervention.
- **Effective Alert System:** LEDs provide visual warnings, while the buzzer provides an audible alert during critical flood conditions.

- **Real-Time Display:** The LCD continuously displays the measured water level and current status.
- **Low Power Requirement:** The Arduino-based prototype requires relatively low power, making it suitable for battery or solar-powered extensions.

Advantages

- Low-cost implementation.
- Non-contact water-level measurement.
- Continuous automatic monitoring.
- Easy to program and modify.
- Immediate visual and audible warning.
- Suitable for laboratory and prototype demonstrations.
- Can be integrated with IoT technologies.
- Thresholds can be customized according to the application.
- Useful for reservoirs, tanks, drainage systems, and flood-prone areas.

Limitations

- HC-SR04 performance can be affected by heavy rain, splashing, foam, turbulence, and condensation.
- The sensor has a limited measurement range.
- A local buzzer cannot alert people who are far away.
- The basic Arduino prototype does not provide remote communication.
- Outdoor deployment requires waterproof housing and appropriate environmental protection.
- A single ultrasonic sensor does not provide redundancy for critical infrastructure.

